

✂ 各组任务说明书 (Task Specifications)

模块 A：无感采集与客户端 (The Daemon)

核心职责：在学生的 Ubuntu 终端后台静默运行，监控作业目录，一旦检测到作业完成，自动打包并向服务器发起提交，全程不需要学生敲击 `submit` 命令。

- **输入：**学生在本地工作区 (Workspace) 创建、修改、保存作业文件。
- **输出：**生成一个标准化的数据包 (作业文件压缩包 + 包含学号、作业ID、MD5校验码的 JSON 文件)，推送到模块 B 的接口。
- **研发要点：**
 - 编写 Linux 守护进程 (基于 `systemd` 或纯后台脚本)。
 - 利用 `inotifywait` 监控文件变动。
 - **极度重要：**必须实现“防抖” (Debounce)。学生使用 Vim 或其他编辑器频繁保存时，不能每次都触发上传。必须设定逻辑 (例如：文件停止变动 3 秒后) 再执行打包上传。
- **独立开发方法：**不需要等服务器端写好。把“上传接口”替换成将打包好的文件复制到本地的 `/tmp/mock_server/` 文件夹即可进行测试。

模块 B：通信中枢与数据库 (The Engine)

核心职责：系统的“心脏”，负责接收模块 A 传来的数据，进行校验、解包、归档，并在数据库中记录每一次提交的状态流转。

- **输入：**来自模块 A 的上传数据包；来自模块 C 的状态修改请求 (如：批改完成、打回重做)。
- **输出：**向模块 C 提供查询接口 (读取未批改列表)；向底层文件系统写入归档文件。
- **研发要点：**
 - 数据库设计：建立 SQLite 数据库，设计好 `submissions` 表 (字段包含：流水号、学号、MD5、提交时间、当前状态)。
 - 安全性校验：收到文件后，必须重新计算一次 MD5，如果与 JSON 中的记录不符，判定为传输损坏，拒绝入库。
 - 并发处理：考虑到可能多人同时提交，确保 SQLite 的写入锁机制正常工作。
- **独立开发方法：**不需要等客户端和前端写好。自己手写几个 JSON 报文和伪造的压缩包，直接往自己的接收脚本里塞，看数据库和文件系统是否正常更新。

模块 C：审批终端与反馈闭环 (The Controller)

核心职责：为助教/教师提供一个纯命令行的交互式界面 (TUI)，用于浏览待批改作业、打分，并自动生成反馈报告推送给学生。

- **输入：**从模块 B 的数据库读取“待批改”状态的记录。
- **输出：**更新数据库状态为“已批改”；生成一份 Markdown 格式的评语反馈文件，并触发推送机制发回给模块 A。
- **研发要点：**
 - 终端 UI 开发：不写网页，使用 Python 的 `Textual` 或 `Rich` 库画出终端界面。左侧作业列表，右侧详

细信息。

- 快捷操作：支持纯键盘流操作（例如绑定 Vim 风格的 `j/k` 上下翻页，`Enter` 确认批改）。
- 反馈生成：将教师输入的分数和评语，自动渲染成排版优雅的 Markdown 文件。
- **独立开发方法**：不需要连真实的数据库。自己建一个包含几十条假数据的 SQLite 文件，直接在上面开发 UI 的增删改查逻辑。

模块 D：异常兜底与一键部署 (The Shield/QA)

核心职责：系统的破坏者和打包者。确保系统不仅在正常情况下能跑，在断网、乱输命令时也不会崩溃，并负责最终的工程化交付。

- **输入**：模块 A、B、C 的所有源码。
- **输出**：自动化测试报告；一键安装配置的 Makefile。
- **研发要点**：
 - 混沌工程：编写“破坏脚本”。模拟断网测试（文件传一半杀掉进程）、注入脏数据（学号输入中文或带空格的特殊字符，如 `test rm -rf.py`）。
 - 日志系统兜底：规范化所有模块的日志输出格式（统一写到 `/var/log/assignment.log`），方便验收时排错。
 - 一键化验收：编写 `Makefile`，确保最终验收时，只需输入 `make install` 就能自动配好环境、建好数据库并拉起所有服务。

配合与防翻车指南（集成验收的关键）

两周不见面各自写代码，很容易在最后一天集成时全面崩溃。必须严格执行以下三点：

1. 第一天只干一件事：定义“API 契约”

- 全组开会，把数据交换的 JSON 结构一字不差地定下来。
- 例如：`{ "student_id": "2024001", "task_id": "A1", "status": "pending", "hash": "abc123xxx" }`。
- 一旦敲定，任何人不准私自修改字段名。即使功能没写完，也要按照这个格式返回假数据。

2. 绝对的路径隔离

- 所有人在代码中**严禁出现**类似 `/home/username/project` 这样的硬编码路径。
- 所有文件读写必须基于相对路径，或者统一从一个配置文件（如 `.env`）中读取根目录变量。

3. 统一部署环境

- 强制所有人使用统一版本的 Linux（推荐 Ubuntu）和 Git 进行版本控制。两周内通过 Git 合并代码，尽早解决代码冲突。